

Learning Objectives

By the end of this session, you will understand:

1. What is **Azure Key Vault**
 2. Why it is used (real-world **use cases**)
 3. Key Vault components: **Secrets, Keys, Certificates**
 4. **Step-by-step demo** using Azure Portal
 5. **Azure CLI example** for automation
 6. Best practices for enterprise security
-

1. What is Azure Key Vault?

Conceptually:

Azure Key Vault is a **cloud service for securely storing and managing sensitive information** such as:

- Application **secrets** (e.g., connection strings, API keys, passwords)
- **Encryption keys** (used for data encryption/decryption)
- **Certificates** (SSL/TLS certificates)
- **Managed identities** and access control for secrets

It acts as a **centralized, secure vault** that helps developers and organizations:

- Avoid storing secrets in **code or configuration files**
 - Control and audit access to secrets
 - Automate **key rotation and versioning**
-

2. Why We Need Key Vault — Practical Use Cases

Use Case	Description
 Store database connection strings	Store SQL connection strings securely and retrieve them at runtime instead of in appsettings.json.

Use Case	Description
 Manage API keys & tokens	Securely store external service keys (e.g., Stripe, Twilio, OpenAI).
 Encrypt application data	Use keys in the vault to encrypt/decrypt data at rest.
 SSL/TLS Certificates management	Automatically renew and manage HTTPS certificates.
 Integrate with Azure services	Access secrets from Azure Functions, App Service, or Virtual Machines using Managed Identity (no credentials).
 Compliance & auditing	Centralized logging of secret access and modification.

3. Key Concepts / Components

1 Secrets

- Store sensitive information (like passwords, API keys).
- Example: "SQLPassword": "MyStrongPass@123"
- Versioning is automatic (each update creates a new version).

2 Keys

- Cryptographic keys for encryption/decryption, signing, etc.
- Example: Encrypt customer data using a Key Vault-managed key.

3 Certificates

- Store and manage SSL/TLS certificates used in web apps.
 - Can integrate with **Azure App Service** for automatic binding.
-

4. Step-by-Step Demo (Using Azure Portal)

Let's walk through creating and using a Key Vault.

Step 1: Create a Key Vault

1. Go to **Azure Portal** → **Create a resource** → Search “**Key Vault**”
2. Click **Create**
3. Fill details:
 - Subscription: <your subscription>
 - Resource Group: demo-rg
 - Key Vault Name: demo-keyvault-001
 - Region: East US
4. Leave default access settings → **Review + Create**

 **Key Vault created!**

Step 2: Add a Secret

1. Inside your Key Vault → Go to **Objects** → **Secrets**
2. Click + **Generate/Import**
3. Name: DbConnectionString
4. Value: Server=tcp:mydb.database.windows.net;Database=DemoDB;User Id=admin;Password=P@ssw0rd;
5. Click **Create**

 Your secret is securely stored!

Step 3: Give Access Permissions

1. Go to **Access control (IAM)** or **Access policies**
 2. Add your user or App Service with “Get” permission for secrets.
- ♦ In new versions, **Azure RBAC** is preferred instead of old access policies.
-

Step 4: Access from an App Service

If you have an **Azure Web App**:

- Assign a **Managed Identity** (System-assigned)
- In Key Vault, grant that identity “Get” permission on secrets
- In your app code, use this format:

```
using Azure.Identity;
```

```
using Azure.Security.KeyVault.Secrets;
```

```
var keyVaultUrl = "https://demo-keyvault-001.vault.azure.net/";
```

```
var client = new SecretClient(new Uri(keyVaultUrl), new DefaultAzureCredential());
```

```
KeyVaultSecret secret = await client.GetSecretAsync("DbConnectionString");
```

```
string connectionString = secret.Value;
```

✅ No password in code — secure and managed automatically!

5. Azure CLI Example

You can do the same using CLI:

Step 1: Create Resource Group

```
az group create --name demo-rg --location eastus
```

Step 2: Create Key Vault

```
az keyvault create --name demo-keyvault-001 --resource-group demo-rg --location eastus
```

Step 3: Add a Secret

```
az keyvault secret set --vault-name demo-keyvault-001 --name "DbConnectionString" --value  
"Server=tcp:mydb.database.windows.net;Database=DemoDB;User  
Id=admin;Password=P@ssw0rd;"
```

Step 4: Get a Secret

```
az keyvault secret show --name "DbConnectionString" --vault-name demo-keyvault-001
```

6. Best Practices

- ✓ **Never hard-code credentials**
 - ✓ Use **Managed Identity** for authentication
 - ✓ Enable **Soft Delete** and **Purge Protection**
 - ✓ Apply **RBAC** for fine-grained access control
 - ✓ Enable **Key rotation** for keys and secrets
 - ✓ Use **Private Endpoints** for secure access (no public internet)
-

7. Real-World Example

Let's say you have a **.NET Web API** hosted in Azure App Service.

- You store DB password in **Key Vault** instead of appsettings.json.
 - Your App Service uses **Managed Identity** to fetch the secret at runtime.
 - You rotate the password every month — no code change needed.
 - Developers never see the real password — reducing human risk.
-

Summary

Concept	Explanation
Azure Key Vault	Central service for managing secrets, keys, and certificates
Secrets	Store sensitive values (passwords, tokens)
Keys	Perform encryption/decryption
Certificates	Manage SSL/TLS securely
Integration	Works with App Service, Functions, Logic Apps, etc.

Concept	Explanation
Security	Controlled via RBAC & Managed Identity